MODULE *protocol*

Version 2 *Zcash P2P* protocol specification, following the draft *ZIP* "Version 2 *Zcash P2P* Network Protocol" (zcash/*zips* $\neq$ 1344).

Models connection setup and block synchronization over the stream layer of *streams.tla*: the *init* handshake on the dedicated handshake stream, protocol version negotiation, the long-lived block announcement streams (including the rule that at most one announcement stream of a type may be open per direction and the sender's option to replace a finished or reset stream), and headers-first synchronization over get-headers and get-blocks request streams, each carrying exactly one request and its response.

Blocks are heights on one linear chain: only the peer at the highest height extends it, and a peer that is behind downloads the heights it lacks.

Communication keeps the legacy model's message consumption discipline: a peer decides only from its own records; the only place both peers' records are written together is the transport itself (connection open/close, stream open, and the freeing of a stream slot once both sides have closed it).

Two boolean constants select how a receiver interprets the draft:

- *StrictSingleton*
  - TRUE a second open announcement stream of the same type from a peer is a *PROTOCOL_ERROR* (literal reading of "Announcement Streams")
  - FALSE the receiver accepts it and drains both
- *RefusePreHandshake*
  - TRUE announcement streams that arrive before the local handshake completes are refused with *REFUSED* ("Connection Handshake": MAY refuse)
  - FALSE they are buffered until it completes

Peers in *ByzantinePeers* complete an honest handshake and may then commit wire-level mischief: a second *init* record, a stream of unknown type, a record of unknown kind on the handshake stream, and a stream finished before its type byte. The receiver's obligations differ per the draft: the first and last are connection errors, the middle two MUST be tolerated (forward compatibility). Ghost flags record genuine violations so that *CloseAccountable* can check no honest receiver ever fires *PROTOCOL_ERROR* without one, and *EventuallyPunished* that every violation ends the connection.

Under *StrictSingleton* the invariant *NoHonestProtocolError* is violated: two conformant peers disconnect because stream independence lets a replacement stream's type byte arrive before the old stream's *FIN* or reset.

See documents/v2-*modeling.md* for the full write-up.

EXTENDS *TLC*, *Naturals*, *Sequences*, *FiniteSets*, *streams*, *records*

CONSTANT *InitialPeers*	set of peers
CONSTANT *MaxBlock*	maximum block height (initial and mined)
CONSTANT *MaxRestarts*	finish/reset of own announcement streams, per peer pair
CONSTANT *MaxHeaders*	headers per get-headers response (draft: 160)
CONSTANT *MaxBlocksPerRequest*	hashes per get-blocks request (draft: 128)
CONSTANT *MinVersion*	minimum protocol version of the draft (not yet assigned)
CONSTANT *Versions*	protocol versions a peer may advertise
CONSTANT *StrictSingleton*	see module comment
CONSTANT *RefusePreHandshake*	see module comment
CONSTANT *ByzantinePeers*	peers that may misbehave after an honest handshake
CONSTANT *MaxMischief*	Byzantine actions available per peer pair

CONSTANT $PunishUnknownType$ TRUE closes on unknown stream types (buggy)

VARIABLE $nodes$

$vars \triangleq \langle nodes \rangle$

See *README* for an explanation of symmetry reduction.
$Symmetry \triangleq Permutations(InitialPeers)$

For each initial peer construct a set of all other peers.
$OtherPeers \triangleq [n \in InitialPeers \mapsto InitialPeers \setminus \{n\}]$

$Min(a, b) \triangleq \text{IF } a < b \text{ THEN } a \text{ ELSE } b$
$Max(a, b) \triangleq \text{IF } a > b \text{ THEN } a \text{ ELSE } b$

$Init \triangleq$
$\quad \exists\, blockset \in [InitialPeers \to (1 \,..\, MaxBlock)] :$
$\quad \exists\, verset \in [InitialPeers \to Versions] :$
$\quad\quad nodes = [i \in InitialPeers \mapsto [$
$\quad\quad\quad conn \mapsto [j \in OtherPeers[i] \mapsto \text{“none”}],$
$\quad\quad\quad close \mapsto [j \in OtherPeers[i] \mapsto NoCode],$
$\quad\quad\quad streams \mapsto [j \in OtherPeers[i] \mapsto [sid \in StreamIds(i, j) \mapsto NullStream]],$
$\quad\quad\quad init\_sent \mapsto [j \in OtherPeers[i] \mapsto \text{FALSE}],$
$\quad\quad\quad init\_recvd \mapsto [j \in OtherPeers[i] \mapsto \text{FALSE}],$
$\quad\quad\quad version \mapsto [j \in OtherPeers[i] \mapsto 0],$
$\quad\quad\quad peer\_tip \mapsto [j \in OtherPeers[i] \mapsto 0],$
$\quad\quad\quad announced \mapsto [j \in OtherPeers[i] \mapsto 0],$
$\quad\quad\quad restarts \mapsto [j \in OtherPeers[i] \mapsto 0],$
$\quad\quad\quad want \mapsto [j \in OtherPeers[i] \mapsto \langle\rangle],$
$\quad\quad\quad score \mapsto [j \in OtherPeers[i] \mapsto 0],$
$\quad\quad\quad mischief \mapsto [j \in OtherPeers[i] \mapsto 0],$
$\quad\quad\quad violated \mapsto [j \in OtherPeers[i] \mapsto \text{FALSE}],$
$\quad\quad\quad blocks \mapsto 1 \,..\, blockset[i],$
$\quad\quad\quad my\_version \mapsto verset[i]$
$\quad\quad]]$

—— Helpers ——

$Height(n) \triangleq Cardinality(nodes[n].blocks)$

$Connected(n, m) \triangleq nodes[n].conn[m] \in \{\text{“initiator”}, \text{“responder”}\}$

The handshake is complete for n once it has both sent and received an *init* record (draft: “Handshake Sequence”, step 3).
$HandshakeComplete(n, m) \triangleq nodes[n].init\_sent[m] \land nodes[n].init\_recvd[m]$

$S(n,\ m,\ sid) \triangleq nodes[n].streams[m][sid]$

Slots n may use to open a new stream toward m (lowest free slot is taken).

$FreeSlots(n,\ m) \triangleq \{k \in 1\ ..\ MaxStreams : S(n,\ m,\ \langle n,\ k\rangle).status = \text{“none”}\}$

$FreeSlot(n,\ m) \triangleq \text{CHOOSE } k \in FreeSlots(n,\ m) : \forall\, j \in FreeSlots(n,\ m) : k \leq j$

Announcement streams of type t that n currently sends to m on.

$$\begin{aligned} LiveAnn(n,\ m,\ t) \triangleq\ & \{sid \in StreamIds(n,\ m) : \\ & \land\ sid[1] = n \\ & \land\ S(n,\ m,\ sid).status = \text{“open”} \\ & \land\ S(n,\ m,\ sid).rtype = t \\ & \land\ S(n,\ m,\ sid).out = \text{“open”}\} \end{aligned}$$

Request streams n has outstanding toward m.

$$\begin{aligned} OpenReq(n,\ m) \triangleq\ & \{sid \in StreamIds(n,\ m) : \\ & \land\ sid[1] = n \\ & \land\ S(n,\ m,\ sid).status = \text{“open”} \\ & \land\ S(n,\ m,\ sid).rtype \in RequestTypes\} \end{aligned}$$

The sequence of heights $a\ ..\ b$ (empty when $b < a$).

$Range(a,\ b) \triangleq [i \in 1\ ..\ (b - a + 1) \mapsto a + i - 1]$

$Contiguous(seq) \triangleq \forall\, i \in 1\ ..\ (Len(seq) - 1) : seq[i + 1] = seq[i] + 1$

Heights of seq above h.

$Above(seq,\ h) \triangleq SelectSeq(seq,\ \text{LAMBDA } x : x > h)$

Handshake streams n knows about on its connection with m.

$$\begin{aligned} HandshakeStreams(n,\ m) \triangleq\ & \{sid \in StreamIds(n,\ m) : \\ & \land\ S(n,\ m,\ sid).status = \text{“open”} \\ & \land\ S(n,\ m,\ sid).rtype = HandshakeType\} \end{aligned}$$

Transport: once both peers have closed a stream its slot is freed.

$$\begin{aligned} & Settle(ns,\ n,\ m,\ sid) \triangleq \\ & \quad \text{IF } \land\ ns[n].streams[m][sid].status = \text{“closed”} \\ & \qquad\ \land\ ns[m].streams[n][sid].status = \text{“closed”} \\ & \quad\ \text{THEN } [ns \text{ EXCEPT } ![n].streams[m][sid] = NullStream, \\ & \qquad\qquad\qquad\qquad\ ![m].streams[n][sid] = NullStream] \\ & \quad\ \text{ELSE } ns \end{aligned}$$

Transport: n closes the connection to m with an application error code.
Both peers observe the close; data in flight is lost.

$$\begin{aligned} & Close(ns,\ n,\ m,\ code) \triangleq \\ & \quad [ns \text{ EXCEPT } ![n].conn[m] \quad\ = \text{“closed”}, \\ & \qquad\qquad\ ![n].close[m] \quad\ = code, \\ & \qquad\qquad\ ![n].streams[m] = [sid \in StreamIds(n,\ m) \mapsto NullStream], \\ & \qquad\qquad\ ![m].conn[n] \quad\ = \text{“closed”}, \\ & \qquad\qquad\ ![m].close[n] \quad\ = code, \end{aligned}$$

$$![m].streams[n] = [sid \in StreamIds(n,\ m) \mapsto NullStream]]$$

— Connection —

n dials m. The *QUIC* handshake is out of scope and establishment is atomic: n becomes the initiator and m the responder of the application handshake.

$$
\begin{array}{l}
Connect \triangleq \\
\quad \exists\, n \in InitialPeers : \\
\quad\quad \exists\, m \in OtherPeers[n] : \\
\quad\quad\quad \land\ nodes[n].conn[m] = \text{“none”} \\
\quad\quad\quad \land\ nodes[m].conn[n] = \text{“none”} \\
\quad\quad\quad \land\ nodes' = [nodes \text{ EXCEPT } ![n].conn[m] = \text{“initiator”}, \\
\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad ![m].conn[n] = \text{“responder”}]
\end{array}
$$

— Handshake —

The initiator opens the handshake stream and sends its *init* record on it (draft: “Handshake Sequence”, step 1). The type byte and the record are queued together: they travel in order on the same stream.

$$
\begin{array}{l}
OpenHandshakeStream \triangleq \\
\quad \exists\, n \in InitialPeers : \\
\quad\quad \exists\, m \in OtherPeers[n] : \\
\quad\quad\quad \land\ nodes[n].conn[m] = \text{“initiator”} \\
\quad\quad\quad \land\ \neg nodes[n].init\_sent[m] \\
\quad\quad\quad \land\ FreeSlots(n,\ m) \neq \{\} \\
\quad\quad\quad \land\ \text{LET } sid \triangleq \langle n,\ FreeSlot(n,\ m)\rangle \\
\quad\quad\quad\quad \text{IN} \quad nodes' = [nodes \text{ EXCEPT} \\
\quad\quad\quad\quad\quad\quad ![n].streams[m][sid] = OpenerStream(HandshakeType), \\
\quad\quad\quad\quad\quad\quad ![m].streams[n][sid] = PeerStream(HandshakeType, \\
\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad \langle MakeInit(nodes[n].my\_version,\ Height(n))\rangle), \\
\quad\quad\quad\quad\quad\quad ![n].init\_sent[m] \quad = \text{TRUE}]
\end{array}
$$

The responder sends its own *init* record on the handshake stream. It may do so as soon as it has seen the stream’s type byte, without waiting for the initiator’s *init* record (draft: “Handshake Sequence”).

$$
\begin{array}{l}
SendInitResponder \triangleq \\
\quad \exists\, n \in InitialPeers : \\
\quad\quad \exists\, m \in OtherPeers[n] : \\
\quad\quad\quad \exists\, sid \in HandshakeStreams(n,\ m) : \\
\quad\quad\quad\quad \land\ nodes[n].conn[m] = \text{“responder”} \\
\quad\quad\quad\quad \land\ \neg nodes[n].init\_sent[m] \\
\quad\quad\quad\quad \land\ nodes' = [nodes \text{ EXCEPT} \\
\quad\quad\quad\quad\quad\quad ![m].streams[n][sid] = PutData(@,\ MakeInit(nodes[n].my\_version,\ Height(n))),
\end{array}
$$

$$!\,[n].init\_sent[m] \quad = \text{TRUE}]$$

n consumes the type byte of a stream m opened. What happens depends on the type and on n's own state:
$0x00$ from the responder, or a second $0x00$ $\rightarrow$ *PROTOCOL_ERROR* ("Connection Handshake")
any other stream before n's handshake completed $\rightarrow$ refuse with *REFUSED*, or wait (*RefusePreHandshake*)
announcement duplicating an open one of type t $\rightarrow$ *PROTOCOL_ERROR*, or accept (*StrictSingleton*)
otherwise $\rightarrow$ record the type

$$
\begin{array}{l}
RecvTypeByte \triangleq \\
\quad \exists\, n \in InitialPeers : \\
\quad\quad \exists\, m \in OtherPeers[n] : \\
\quad\quad\quad \exists\, sid \in StreamIds(n,\, m) : \\
\quad\quad\quad\quad \text{LET } s \triangleq S(n,\, m,\, sid) \\
\quad\quad\quad\quad \text{IN} \\
\quad\quad\quad\quad \land\, Connected(n,\, m) \\
\quad\quad\quad\quad \land\, s.status = \text{"open"} \\
\quad\quad\quad\quad \land\, s.rtype = \text{"unknown"} \\
\quad\quad\quad\quad \land\, Len(s.inq) > 0 \\
\quad\quad\quad\quad \land\, IsTypeByte(Head(s.inq)) \\
\quad\quad\quad\quad \land\, \text{LET } t \triangleq Head(s.inq).t \\
\quad\quad\quad\quad\quad\quad accept \triangleq [nodes \text{ EXCEPT } ![n].streams[m][sid].rtype = t, \\
\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad ![n].streams[m][sid].inq \;\; = Tail(@)] \\
\quad\quad\quad\quad\quad\quad legalHs \triangleq nodes[n].conn[m] = \text{"responder"} \land HandshakeStreams(n,\, m) = \{\} \\
\quad\quad\quad\quad\quad\quad dup \triangleq \exists\, o \in StreamIds(n,\, m) : \\
\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad \land\, o \neq sid \\
\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad \land\, o[1] = m \\
\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad \land\, S(n,\, m,\, o).status = \text{"open"} \\
\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad \land\, S(n,\, m,\, o).rtype = t \\
\quad\quad\quad\quad \text{IN} \\
\quad\quad\quad\quad \lor\, \land\, t \notin StreamTypes \\
\quad\quad\quad\quad\quad\quad \text{Unknown stream type: refuse with} \\
\quad\quad\quad\quad\quad\quad \textit{UNSUPPORTED\_STREAM\_TYPE}\text{; the draft says MUST NOT} \\
\quad\quad\quad\quad\quad\quad \text{close or penalize — new stream types deploy} \\
\quad\quad\quad\quad\quad\quad \text{without version gating. } \textit{PunishUnknownType} \text{ is the} \\
\quad\quad\quad\quad\quad\quad \text{forbidden reading.} \\
\quad\quad\quad\quad\quad\quad \land\, \lor\, \land\, PunishUnknownType \\
\quad\quad\quad\quad\quad\quad\quad\quad \land\, nodes' = Close(nodes,\, n,\, m,\, \text{"PROTOCOL\_ERROR"}) \\
\quad\quad\quad\quad\quad\quad\quad\quad \lor\, \land\, \neg PunishUnknownType \\
\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad \land\, nodes' = Settle([nodes \text{ EXCEPT} \\
\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad ![n].streams[m][sid] = ClosedStream, \\
\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad ![m].streams[n][sid] = Refuse(@,\, t,\, \text{"UNSUPPORTED\_STREAM\_TYPE"})], \\
\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad n,\, m,\, sid) \\
\quad\quad\quad\quad\quad\quad \lor\, \land\, t \in StreamTypes \\
\quad\quad\quad\quad\quad\quad\quad\quad \land\, \lor\, \land\, t = HandshakeType \\
\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad \land\, \lor\, \land\, legalHs
\end{array}
$$

```
                    ∧ nodes′ = accept
                ∨ ∧ ¬legalHs
                    ∧ nodes′ = Close(nodes, n, m, "PROTOCOL_ERROR")
            ∨ ∧ t ∈ AnnTypes ∪ RequestTypes
              ∧ ∨ ∧ ¬HandshakeComplete(n, m)
                    ∧ RefusePreHandshake
                    ∧ nodes′ = Settle([nodes EXCEPT
                                ![n].streams[m][sid] = ClosedStream,
                                ![m].streams[n][sid] = Refuse(@, t, "REFUSED")],
                              n, m, sid)
                ∨ ∧ HandshakeComplete(n, m)
                  ∧ ∨ ∧ t ∈ AnnTypes ∧ StrictSingleton ∧ dup
                        ∧ nodes′ = Close(nodes, n, m, "PROTOCOL_ERROR")
                    ∨ ∧ ¬(t ∈ AnnTypes ∧ StrictSingleton ∧ dup)
                        ∧ nodes′ = accept
```

n consumes m's *init* record from the handshake stream (draft: "Handshake Validation"):
- second *init* record → *PROTOCOL_ERROR*
- version below *MinVersion* → *OBSOLETE*
- otherwise → handshake received; negotiated *version* = *min*

```
RecvInit ≜
  ∃ n ∈ InitialPeers :
    ∃ m ∈ OtherPeers[n] :
      ∃ sid ∈ HandshakeStreams(n, m) :
        LET s ≜ S(n, m, sid)
        IN
         ∧ Connected(n, m)
         ∧ Len(s.inq) > 0
         ∧ Head(s.inq).kind = "init"
         ∧ LET msg ≜ Head(s.inq)
           IN
            ∨ ∧ nodes[n].init_recvd[m]
              ∧ nodes′ = Close(nodes, n, m, "PROTOCOL_ERROR")
            ∨ ∧ ¬nodes[n].init_recvd[m]
              ∧ msg.version < MinVersion
              ∧ nodes′ = Close(nodes, n, m, "OBSOLETE")
            ∨ ∧ ¬nodes[n].init_recvd[m]
              ∧ msg.version ≥ MinVersion
              ∧ nodes′ = [nodes EXCEPT
                    ![n].streams[m][sid].inq = Tail(@),
                    ![n].init_recvd[m]       = TRUE,
                    ![n].version[m]          = Min(nodes[n].my_version, msg.version),
                    ![n].peer_tip[m]         = msg.start_height]
```

n ignores a record of unknown kind on the handshake stream: "a node MUST ignore handshake-stream records whose kind it does not recognize" (draft: "Connection Handshake").

$$
\begin{array}{l}
RecvUnknownHandshakeRecord \triangleq \\
\quad \exists\, n \in InitialPeers : \\
\quad\quad \exists\, m \in OtherPeers[n] : \\
\quad\quad\quad \exists\, sid \in HandshakeStreams(n,\, m) : \\
\quad\quad\quad\quad \text{LET } s \triangleq S(n,\, m,\, sid) \\
\quad\quad\quad\quad \text{IN} \\
\quad\quad\quad\quad \land\, Connected(n,\, m) \\
\quad\quad\quad\quad \land\, Len(s.inq) > 0 \\
\quad\quad\quad\quad \land\, s.inq[1].kind \notin \{\text{“init”},\ \text{“type”},\ \text{“fin”}\} \\
\quad\quad\quad\quad \land\, nodes' = [nodes \text{ EXCEPT } ![n].streams[m][sid].inq = Tail(@)]
\end{array}
$$

— *Byzantine* actions —

A *Byzantine* peer completes an honest handshake and then misbehaves at the wire level, within a *MaxMischief* budget. Ghost bookkeeping: the HONEST side's *violated flag records genuine violations(the first and last are; the tolerated two are not), so accountability is checkable.*

$$
\begin{array}{l}
ByzCan(n,\, m) \triangleq \\
\quad \land\, n \in ByzantinePeers \\
\quad \land\, Connected(n,\, m) \\
\quad \land\, HandshakeComplete(n,\, m) \\
\quad \land\, nodes[n].mischief[m] < MaxMischief
\end{array}
$$

A second init record on the handshake stream | a violation the receiver MUST answer with PROTOCOL_ERROR(draft : "Handshake Validation").

$$
\begin{array}{l}
ByzSecondInit \triangleq \\
\quad \exists\, n \in InitialPeers : \\
\quad\quad \exists\, m \in OtherPeers[n] : \\
\quad\quad\quad \exists\, sid \in HandshakeStreams(n,\, m) : \\
\quad\quad\quad\quad \land\, ByzCan(n,\, m) \\
\quad\quad\quad\quad \land\, nodes' = [nodes \text{ EXCEPT} \\
\quad\quad\quad\quad\quad\quad ![m].streams[n][sid] = PutData(@,\, MakeInit(nodes[n].my\_version,\, Height(n))), \\
\quad\quad\quad\quad\quad\quad ![n].mischief[m] \quad\ = @ + 1, \\
\quad\quad\quad\quad\quad\quad ![m].violated[n] \quad\ = \text{TRUE}]
\end{array}
$$

A record of unknown kind on the handshake stream | NOT a violation : future revisions may define new kinds and the receiver MUST ignore them.

$$
\begin{array}{l}
ByzUnknownHandshakeRecord \triangleq \\
\quad \exists\, n \in InitialPeers : \\
\quad\quad \exists\, m \in OtherPeers[n] : \\
\quad\quad\quad \exists\, sid \in HandshakeStreams(n,\, m) :
\end{array}
$$

$\land\ ByzCan(n,\ m)$
$\land\ nodes' = [nodes\ \text{EXCEPT}$
$\quad ![m].streams[n][sid] = PutData(@,\ [kind \mapsto \text{“junk”}]),$
$\quad ![n].mischief[m] \quad = @ + 1]$

A stream of a type the receiver does not recognize | NOT a violation :
the receiver refuses it with UNSUPPORTED_STREAM_TYPE and moves on
(*draft* : “Stream Types”).

$ByzUnknownStreamType \triangleq$
$\exists\, n \in InitialPeers :$
$\exists\, m \in OtherPeers[n] :$
$\land\ ByzCan(n,\ m)$
$\land\ FreeSlots(n,\ m) \neq \{\}$
$\land\ \text{LET}\ sid \triangleq \langle n,\ FreeSlot(n,\ m)\rangle$
$\text{IN}\quad nodes' = [nodes\ \text{EXCEPT}$
$\quad ![n].streams[m][sid] = ClosedStream,$
$\quad ![m].streams[n][sid] = PeerStream(\text{“0xEE”},\ \langle\rangle),$
$\quad ![n].mischief[m] \quad = @ + 1]$

A handshake stream opened by a peer that already has one(or is the
responder)| a violation the receiver MUST answer with PROTOCOL_ERROR
(*draft* : “Connection Handshake”).

$ByzSecondHandshakeStream \triangleq$
$\exists\, n \in InitialPeers :$
$\exists\, m \in OtherPeers[n] :$
$\land\ ByzCan(n,\ m)$
$\land\ FreeSlots(n,\ m) \neq \{\}$
$\land\ \text{LET}\ sid \triangleq \langle n,\ FreeSlot(n,\ m)\rangle$
$\text{IN}\quad nodes' = [nodes\ \text{EXCEPT}$
$\quad ![n].streams[m][sid] = ClosedStream,$
$\quad ![m].streams[n][sid] = PeerStream(HandshakeType,\ \langle\rangle),$
$\quad ![n].mischief[m] \quad = @ + 1,$
$\quad ![m].violated[n] \quad = \text{TRUE}]$

A stream finished before a complete type byte | a violation the receiver
MUST answer with PROTOCOL_ERROR(*draft* : “Stream Types”).

$ByzEarlyFin \triangleq$
$\exists\, n \in InitialPeers :$
$\exists\, m \in OtherPeers[n] :$
$\land\ ByzCan(n,\ m)$
$\land\ FreeSlots(n,\ m) \neq \{\}$
$\land\ \text{LET}\ sid \triangleq \langle n,\ FreeSlot(n,\ m)\rangle$
$\text{IN}\quad nodes' = [nodes\ \text{EXCEPT}$
$\quad ![n].streams[m][sid] = ClosedStream,$
$\quad ![m].streams[n][sid] = [NullStream\ \text{EXCEPT}\ !.status = \text{“open”},$
$\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad !.inq = \langle FIN\rangle],$

$$!\,[n].mischief[m] \quad = @ + 1,$$
$$!\,[m].violated[n] \quad = \text{TRUE}]$$

— Announcement streams —

After its handshake completes, n opens one announcement stream of each type it has none open for(draft : "Announcement Streams"). This is also how a replacement is opened after a finish, reset, or refusal. A reset can overtake announcements still in flight, so the sender forgets what it announced and re − announces its tip on the new stream.

$$OpenAnnouncementStream \triangleq$$
$$\exists\, n \in InitialPeers :$$
$$\exists\, m \in OtherPeers[n] :$$
$$\exists\, t \in AnnTypes :$$
$$\land\ Connected(n,\ m)$$
$$\land\ HandshakeComplete(n,\ m)$$
$$\land\ LiveAnn(n,\ m,\ t) = \{\}$$
$$\land\ FreeSlots(n,\ m) \neq \{\}$$
$$\land \text{ LET } sid \triangleq \langle n,\ FreeSlot(n,\ m)\rangle$$
$$\text{IN} \quad nodes' = [nodes \text{ EXCEPT}$$
$$!\,[n].streams[m][sid] = OpenerStream(t),$$
$$!\,[m].streams[n][sid] = PeerStream(t,\ \langle\rangle),$$
$$!\,[n].announced[m] \ = 0]$$

n announces its current tip to m when it has grown since the last announcement(draft : "Block Announcements", header announcement).

$$SendAnnouncement \triangleq$$
$$\exists\, n \in InitialPeers :$$
$$\exists\, m \in OtherPeers[n] :$$
$$\exists\, sid \in LiveAnn(n,\ m,\ BlockAnnType) :$$
$$\land\ nodes[n].announced[m] < Height(n)$$
$$\land\ nodes' = [nodes \text{ EXCEPT}$$
$$!\,[m].streams[n][sid] = PutData(@,\ MakeHeaderAnnouncement(Height(n))),$$
$$!\,[n].announced[m] \ = Height(n)]$$

n consumes a header announcement from m. Announcements are only processed once n s handshake is complete (draft: "Connection Handshake").

$$RecvAnnouncement \triangleq$$
$$\exists\, n \in InitialPeers :$$
$$\exists\, m \in OtherPeers[n] :$$
$$\exists\, sid \in StreamIds(n,\ m) :$$
$$\text{LET } s \triangleq S(n,\ m,\ sid)$$
$$\text{IN}$$
$$\land\ Connected(n,\ m)$$

$\land\ HandshakeComplete(n,\ m)$
$\land\ s.status = \text{“open”}$
$\land\ s.rtype \in AnnTypes$
$\land\ Len(s.inq) > 0$
$\land\ Head(s.inq).kind = \text{“header”}$
$\land\ nodes' = [nodes \text{ EXCEPT}$
$\quad ![n].streams[m][sid].inq = Tail(@),$
$\quad ![n].peer\_tip[m] = Max(@,\ Head(s.inq).height)]$

n finishes one of its announcement streams. The draft allows a sender to open a replacement afterwards; why it finished (backpressure, internal restart) is not modeled. Bounded by *MaxRestarts* to keep liveness meaningful.

$FinishAnnouncementStream \triangleq$
$\exists\, n \in InitialPeers :$
$\exists\, m \in OtherPeers[n] :$
$\exists\, sid \in LiveAnn(n,\ m,\ BlockAnnType) :$
$\land\ nodes[n].restarts[m] < MaxRestarts$
$\land\ nodes' = Settle([nodes \text{ EXCEPT}$
$\quad ![n].streams[m][sid] = ClosedStream,$
$\quad ![m].streams[n][sid] = PutData(@,\ FIN),$
$\quad ![n].restarts[m] = @ + 1],$
$\quad n,\ m,\ sid)$

Same as *FinishAnnouncementStream* but via *RESET_STREAM*, which can overtake records still in flight on that stream.

$ResetAnnouncementStream \triangleq$
$\exists\, n \in InitialPeers :$
$\exists\, m \in OtherPeers[n] :$
$\exists\, sid \in LiveAnn(n,\ m,\ BlockAnnType) :$
$\land\ nodes[n].restarts[m] < MaxRestarts$
$\land\ nodes' = Settle([nodes \text{ EXCEPT}$
$\quad ![n].streams[m][sid] = ClosedStream,$
$\quad ![m].streams[n][sid] = PutReset(@,\ \text{“INTERNAL\_ERROR”}),$
$\quad ![n].restarts[m] = @ + 1],$
$\quad n,\ m,\ sid)$

— Request streams: headers-first synchronization —

n is behind m (from m's *init* or announcements), has nothing queued to download and no request outstanding: it asks m for headers after its own tip (draft: “Headers-First Synchronization”, step 1). The request and the *FIN* of n's sending direction are written together.

$SendGetHeaders \triangleq$
$\exists\, n \in InitialPeers :$

```
        ∃ m ∈ OtherPeers[n] :
            ∧ Connected(n, m)
            ∧ HandshakeComplete(n, m)
            ∧ nodes[n].peer_tip[m] > Height(n)
            ∧ nodes[n].want[m] = ⟨⟩
            ∧ OpenReq(n, m) = {}
            ∧ FreeSlots(n, m) ≠ {}
            ∧ LET sid ≜ ⟨n, FreeSlot(n, m)⟩
              IN  nodes' = [nodes EXCEPT
                    ![n].streams[m][sid] = RequesterStream(GetHeadersType),
                    ![m].streams[n][sid] = PeerStream(GetHeadersType,
                                                      ⟨MakeGetHeaders(Height(n)), FIN⟩)]
```

n serves a get-headers request from m: the headers after the locator, at most *MaxHeaders* of them, then *FIN*. Serving may start as soon as the request is complete, before m's *FIN* has been consumed (draft: "Request Streams").

```
ServeGetHeaders ≜
    ∃ n ∈ InitialPeers :
      ∃ m ∈ OtherPeers[n] :
        ∃ sid ∈ StreamIds(n, m) :
          LET s ≜ S(n, m, sid)
          IN
           ∧ Connected(n, m)
           ∧ HandshakeComplete(n, m)
           ∧ s.status = "open"
           ∧ s.rtype = GetHeadersType
           ∧ s.out = "open"
           ∧ Len(s.inq) > 0
           ∧ Head(s.inq).kind = "get-headers"
           ∧ LET loc  ≜ Head(s.inq).locator
                 hdrs ≜ Range(loc + 1, Min(loc + MaxHeaders, Height(n)))
             IN  nodes' = [nodes EXCEPT
                   ![n].streams[m][sid] = Collapse([@ EXCEPT !.inq = Tail(@),
                                                            !.out = "finished"]),
                   ![m].streams[n][sid] = PutData(PutData(@, MakeHeaders(hdrs)), FIN)]
```

n consumes a headers response from m (draft: "get-headers"):
more than *MaxHeaders*, or not contiguous → discard, misbehavior penalty
otherwise → queue the heights n lacks

```
RecvHeaders ≜
    ∃ n ∈ InitialPeers :
      ∃ m ∈ OtherPeers[n] :
        ∃ sid ∈ OpenReq(n, m) :
          LET s ≜ S(n, m, sid)
```

```
            IN
             ∧ Connected(n, m)
             ∧ s.rtype = GetHeadersType
             ∧ Len(s.inq) > 0
             ∧ Head(s.inq).kind = "headers"
             ∧ LET hdrs ≜ Head(s.inq).heights
               IN
                ∨ ∧ Len(hdrs) ≤ MaxHeaders ∧ Contiguous(hdrs)
                  ∧ nodes' = [nodes EXCEPT
                              ![n].streams[m][sid].inq = Tail(@),
                              ![n].want[m]             = Above(hdrs, Height(n))]
                ∨ ∧ ¬(Len(hdrs) ≤ MaxHeaders ∧ Contiguous(hdrs))
                  ∧ nodes' = [nodes EXCEPT
                              ![n].streams[m][sid].inq = Tail(@),
                              ![n].score[m]            = @ + 20]
```

n requests the next batch of blocks it wants from m, at most *MaxBlocksPerRequest* of them (draft: "get-blocks").

```
SendGetBlocks ≜
  ∃ n ∈ InitialPeers :
    ∃ m ∈ OtherPeers[n] :
      ∧ Connected(n, m)
      ∧ nodes[n].want[m] ≠ ⟨⟩
      ∧ OpenReq(n, m) = {}
      ∧ FreeSlots(n, m) ≠ {}
      ∧ LET sid   ≜ ⟨n, FreeSlot(n, m)⟩
            batch ≜ SubSeq(nodes[n].want[m], 1,
                           Min(MaxBlocksPerRequest, Len(nodes[n].want[m])))
        IN  nodes' = [nodes EXCEPT
              ![n].streams[m][sid] = RequesterStream(GetBlocksType),
              ![m].streams[n][sid] = PeerStream(GetBlocksType,
                                                ⟨MakeGetBlocks(batch), FIN⟩)]
```

n serves a get-blocks request from m. It may finish after any complete entry, delivering fewer blocks than requested (draft: "get-blocks"); the requester re-requests the remainder.

```
ServeGetBlocks ≜
  ∃ n ∈ InitialPeers :
    ∃ m ∈ OtherPeers[n] :
      ∃ sid ∈ StreamIds(n, m) :
        LET s ≜ S(n, m, sid)
        IN
         ∧ Connected(n, m)
         ∧ HandshakeComplete(n, m)
         ∧ s.status = "open"
```

```
                ∧ s.rtype = GetBlocksType
                ∧ s.out = "open"
                ∧ Len(s.inq) > 0
                ∧ Head(s.inq).kind = "get-blocks"
                ∧ LET asked ≜ Head(s.inq).heights
                      held  ≜ SelectSeq(asked, LAMBDA h : h ∈ nodes[n].blocks)
                  IN
                  ∃ k ∈ 1 .. Len(held) :
                    nodes' = [nodes EXCEPT
                          ![n].streams[m][sid] = Collapse([@ EXCEPT !.inq = Tail(@),
                                                                   !.out = "finished"]),
                          ![m].streams[n][sid] = PutData(PutData(@, MakeBlocks(SubSeq(held, 1, k))), FIN)]
```

n consumes delivered blocks from m and extends its chain with them.

```
RecvBlocks ≜
    ∃ n ∈ InitialPeers :
      ∃ m ∈ OtherPeers[n] :
        ∃ sid ∈ OpenReq(n, m) :
          LET s ≜ S(n, m, sid)
          IN
           ∧ Connected(n, m)
           ∧ s.rtype = GetBlocksType
           ∧ Len(s.inq) > 0
           ∧ Head(s.inq).kind = "blocks"
           ∧ LET got    ≜ Head(s.inq).heights
                 blocks ≜ nodes[n].blocks ∪ {got[i] : i ∈ 1 .. Len(got)}
             IN  nodes' = [nodes EXCEPT
                   ![n].streams[m][sid].inq = Tail(@),
                   ![n].blocks              = blocks,
                   ![n].want[m]             = Above(nodes[n].want[m], Cardinality(blocks))]
```

— Stream teardown —

n consumes a *FIN*: the peer's sending direction is over, and the stream closes once n's own direction is too. On a request stream the *FIN* may arrive after the response was already served; it is never "data after a complete request". A *FIN* before the type byte, or on the handshake stream, is handled per the draft (*PROTOCOL_ERROR*; graceful close) although honest peers never produce either.

```
RecvFin ≜
    ∃ n ∈ InitialPeers :
      ∃ m ∈ OtherPeers[n] :
        ∃ sid ∈ StreamIds(n, m) :
          LET s ≜ S(n, m, sid)
```

```
            IN
             ∧ Connected(n, m)
             ∧ s.status = "open"
             ∧ Len(s.inq) > 0
             ∧ IsFin(Head(s.inq))
             ∧ ∨ ∧ s.rtype = "unknown"
                   ∧ nodes' = Close(nodes, n, m, "PROTOCOL_ERROR")
               ∨ ∧ s.rtype = HandshakeType
                   ∧ nodes' = Close(nodes, n, m, "NO_ERROR")
               ∨ ∧ s.rtype ∈ AnnTypes ∪ RequestTypes
                   ∧ nodes' = Settle([nodes EXCEPT
                                 ![n].streams[m][sid] = Collapse([@ EXCEPT !.inq = Tail(@),
                                                                          !.in_done = TRUE])],
                              n, m, sid)
```

n observes a *RESET_STREAM* from m. Queued data on that stream is discarded.

```
RecvReset ≜
  ∃ n ∈ InitialPeers :
    ∃ m ∈ OtherPeers[n] :
      ∃ sid ∈ StreamIds(n, m) :
        LET s ≜ S(n, m, sid)
        IN
         ∧ Connected(n, m)
         ∧ s.status = "open"
         ∧ s.in_reset ≠ NoCode
         ∧ ∨ ∧ s.rtype = HandshakeType
               ∧ nodes' = Close(nodes, n, m, "NO_ERROR")
           ∨ ∧ s.rtype ≠ HandshakeType
               ∧ nodes' = Settle([nodes EXCEPT ![n].streams[m][sid] = ClosedStream],
                                 n, m, sid)
```

n observes a *STOP_SENDING* from m on a stream n is sending on, and resets its sending direction as the draft recommends ("Request Streams").

```
RecvStop ≜
  ∃ n ∈ InitialPeers :
    ∃ m ∈ OtherPeers[n] :
      ∃ sid ∈ StreamIds(n, m) :
        LET s ≜ S(n, m, sid)
        IN
         ∧ Connected(n, m)
         ∧ sid[1] = n
         ∧ s.status = "open"
         ∧ s.out = "open"
         ∧ s.stop ≠ NoCode
         ∧ nodes' = Settle([nodes EXCEPT
```

$$
\begin{aligned}
&\quad ![n].streams[m][sid] = ClosedStream,\\
&\quad ![m].streams[n][sid] = PutReset(@,\ s.stop)],\\
&n,\ m,\ sid)
\end{aligned}
$$

— Chain growth —

The peer at the chain tip finds a new block, giving it something to announce. Only the highest peer extends the chain, which keeps the model a single linear chain that the others catch up with.

$$
\begin{aligned}
&MineBlock \triangleq\\
&\quad \exists\, n \in InitialPeers :\\
&\qquad \land\ Height(n) < MaxBlock\\
&\qquad \land\ \forall\, m \in OtherPeers[n] : Height(m) \leq Height(n)\\
&\qquad \land\ nodes' = [nodes \text{ EXCEPT } ![n].blocks = @ \cup \{Height(n) + 1\}]
\end{aligned}
$$

$$
\begin{aligned}
&Next \triangleq\\
&\quad \lor\ Connect\\
&\quad \lor\ OpenHandshakeStream\\
&\quad \lor\ SendInitResponder\\
&\quad \lor\ RecvTypeByte\\
&\quad \lor\ RecvInit\\
&\quad \lor\ RecvUnknownHandshakeRecord\\
&\quad \lor\ ByzSecondInit\\
&\quad \lor\ ByzUnknownHandshakeRecord\\
&\quad \lor\ ByzUnknownStreamType\\
&\quad \lor\ ByzSecondHandshakeStream\\
&\quad \lor\ ByzEarlyFin\\
&\quad \lor\ OpenAnnouncementStream\\
&\quad \lor\ SendAnnouncement\\
&\quad \lor\ RecvAnnouncement\\
&\quad \lor\ FinishAnnouncementStream\\
&\quad \lor\ ResetAnnouncementStream\\
&\quad \lor\ SendGetHeaders\\
&\quad \lor\ ServeGetHeaders\\
&\quad \lor\ RecvHeaders\\
&\quad \lor\ SendGetBlocks\\
&\quad \lor\ ServeGetBlocks\\
&\quad \lor\ RecvBlocks\\
&\quad \lor\ RecvFin\\
&\quad \lor\ RecvReset\\
&\quad \lor\ RecvStop\\
&\quad \lor\ MineBlock
\end{aligned}
$$

Weak fairness on *Next*: as long as any action is enabled, one is taken. Mining and restarts are bounded, so every behaviour quiesces and every enabled receive eventually happens. The one loop that is not bounded is a responder refusing pre-handshake announcement streams while the sender keeps reopening them; fairness on *RecvInit* guarantees the responder eventually processes the *init* record that ends that loop.

$$
\begin{aligned}
&Spec \triangleq \\
&\quad Init \\
&\quad \land \Box[Next]_{vars} \\
&\quad \land \mathrm{WF}_{vars}(Next) \\
&\quad \land \mathrm{WF}_{vars}(RecvInit)
\end{aligned}
$$

── Liveness ──

Every connection eventually completes its handshake (or has been closed).

$$
\begin{aligned}
&HandshakeCompletes \triangleq \\
&\quad \Diamond\Box \forall\, n \in InitialPeers : \forall\, m \in OtherPeers[n] : \\
&\qquad nodes[n].conn[m] = \text{“closed”} \lor HandshakeComplete(n,\ m)
\end{aligned}
$$

Every peer eventually learns every connected peer's final tip.

$$
\begin{aligned}
&AnnouncementsFlow \triangleq \\
&\quad \Diamond\Box \forall\, n \in InitialPeers : \forall\, m \in OtherPeers[n] : \\
&\qquad nodes[n].conn[m] = \text{“closed”} \lor nodes[n].peer\_tip[m] = Height(m)
\end{aligned}
$$

Eventually all peers hold the same chain.

$$
\begin{aligned}
&EventualConsensus \triangleq \\
&\quad \Diamond\Box \forall\, i,\ j \in InitialPeers : nodes[i].blocks = nodes[j].blocks
\end{aligned}
$$

── Safety invariants ──

$$
\begin{aligned}
&TypeOK \triangleq \\
&\quad \forall\, n \in InitialPeers : \forall\, m \in OtherPeers[n] : \\
&\qquad \land\ nodes[n].conn[m] \in \{\text{“none”},\ \text{“initiator”},\ \text{“responder”},\ \text{“closed”}\} \\
&\qquad \land\ nodes[n].close[m] \in ErrorCodes \cup \{NoCode\} \\
&\qquad \land\ nodes[n].version[m] \in Versions \cup \{0\} \\
&\qquad \land\ nodes[n].restarts[m] \in 0\,..\,MaxRestarts \\
&\qquad \land\ nodes[n].score[m] \in Nat \\
&\qquad \land\ nodes[n].mischief[m] \in 0\,..\,MaxMischief \\
&\qquad \land\ \forall\, sid \in StreamIds(n,\ m) : \\
&\qquad\quad \land\ S(n,\ m,\ sid).status \in \{\text{“none”},\ \text{“open”},\ \text{“closed”}\} \\
&\qquad\quad \land\ S(n,\ m,\ sid).rtype \in StreamTypes \cup \{\text{“unknown”}\} \\
&\qquad\quad \land\ S(n,\ m,\ sid).out \in \{\text{“open”},\ \text{“finished”},\ \text{“reset”},\ \text{“na”}\}
\end{aligned}
$$

A stream slot is free on one side exactly when it is free on the other.

$$
\begin{array}{l}
\mathit{SlotsConsistent} \triangleq \\
\quad \forall\, n \in \mathit{InitialPeers} : \forall\, m \in \mathit{OtherPeers}[n] : \forall\, \mathit{sid} \in \mathit{StreamIds}(n,\, m) : \\
\qquad (S(n,\, m,\, \mathit{sid}).\mathit{status} = \text{“none”}) = (S(m,\, n,\, \mathit{sid}).\mathit{status} = \text{“none”})
\end{array}
$$

At most one handshake stream per connection, opened by the initiator (draft: “Connection Handshake”).

$$
\begin{array}{l}
\mathit{OneHandshakeStream} \triangleq \\
\quad \forall\, n \in \mathit{InitialPeers} : \forall\, m \in \mathit{OtherPeers}[n] : \\
\qquad \land\ \mathit{Cardinality}(\mathit{HandshakeStreams}(n,\, m)) \leq 1 \\
\qquad \land\ \forall\, \mathit{sid} \in \mathit{HandshakeStreams}(n,\, m) : \\
\qquad\quad \text{IF } \mathit{sid}[1] = n \text{ THEN } \mathit{nodes}[n].\mathit{conn}[m] = \text{“initiator”} \\
\qquad\qquad\qquad\ \text{ELSE } \mathit{nodes}[m].\mathit{conn}[n] = \text{“initiator”}
\end{array}
$$

A peer opens no stream before sending its *init* record, and no announcement stream before its handshake completes (draft: “Connection Handshake”).

$$
\begin{array}{l}
\mathit{HandshakeBeforeStreams} \triangleq \\
\quad \forall\, n \in \mathit{InitialPeers} : \forall\, m \in \mathit{OtherPeers}[n] : \\
\qquad \land\ \neg \mathit{nodes}[n].\mathit{init\_sent}[m] \Rightarrow \\
\qquad\quad \forall\, k \in 1\,..\,\mathit{MaxStreams} : S(n,\, m,\, \langle n,\, k \rangle).\mathit{status} = \text{“none”} \\
\qquad \land\ \neg \mathit{HandshakeComplete}(n,\, m) \Rightarrow \\
\qquad\quad \forall\, k \in 1\,..\,\mathit{MaxStreams} : S(n,\, m,\, \langle n,\, k \rangle).\mathit{rtype} \notin \mathit{AnnTypes}
\end{array}
$$

Once n has received m's *init*, the negotiated version is the minimum of the two advertised versions (draft: “Protocol *Versioning*”).

$$
\begin{array}{l}
\mathit{NegotiatedVersionIsMin} \triangleq \\
\quad \forall\, n \in \mathit{InitialPeers} : \forall\, m \in \mathit{OtherPeers}[n] : \\
\qquad \mathit{HandshakeComplete}(n,\, m) \Rightarrow \\
\qquad\quad \mathit{nodes}[n].\mathit{version}[m] = \mathit{Min}(\mathit{nodes}[n].\mathit{my\_version},\, \mathit{nodes}[m].\mathit{my\_version})
\end{array}
$$

A sender never has two announcement streams of one type open toward a peer (draft: “Announcement Streams”).

$$
\begin{array}{l}
\mathit{SenderSingleton} \triangleq \\
\quad \forall\, n \in \mathit{InitialPeers} : \forall\, m \in \mathit{OtherPeers}[n] : \forall\, t \in \mathit{AnnTypes} : \\
\qquad \mathit{Cardinality}(\mathit{LiveAnn}(n,\, m,\, t)) \leq 1
\end{array}
$$

OBSOLETE is only ever used against a peer that advertised an old version.

$$
\begin{array}{l}
\mathit{CloseJustified} \triangleq \\
\quad \forall\, n \in \mathit{InitialPeers} : \forall\, m \in \mathit{OtherPeers}[n] : \\
\qquad \mathit{nodes}[n].\mathit{close}[m] = \text{“OBSOLETE”} \Rightarrow \\
\qquad\quad \lor\ \mathit{nodes}[n].\mathit{my\_version}\ < \mathit{MinVersion} \\
\qquad\quad \lor\ \mathit{nodes}[m].\mathit{my\_version} < \mathit{MinVersion}
\end{array}
$$

Blocks are always a contiguous chain from genesis.

$$
\begin{array}{l}
\mathit{BlocksContiguous} \triangleq \\
\quad \forall\, n \in \mathit{InitialPeers} : \mathit{nodes}[n].\mathit{blocks} = 1\,..\,\mathit{Height}(n)
\end{array}
$$

Request streams are only opened after the handshake completes, one at a

time per peer (draft: “Connection Handshake”; $ZIP-204$ in-transit limit).

$$
\begin{aligned}
&\mathit{RequestsAfterHandshake} \triangleq \\
&\quad \forall\, n \in \mathit{InitialPeers} : \forall\, m \in \mathit{OtherPeers}[n] : \\
&\qquad \land\ \mathit{OpenReq}(n,\ m) \neq \{\} \Rightarrow \mathit{HandshakeComplete}(n,\ m) \\
&\qquad \land\ \mathit{Cardinality}(\mathit{OpenReq}(n,\ m)) \leq 1
\end{aligned}
$$

Responses in flight respect the draft’s bounds: at most $\mathit{MaxHeaders}$ contiguous headers, at most $\mathit{MaxBlocksPerRequest}$ hashes per get-blocks.

$$
\begin{aligned}
&\mathit{ResponsesBounded} \triangleq \\
&\quad \forall\, n \in \mathit{InitialPeers} : \forall\, m \in \mathit{OtherPeers}[n] : \forall\, \mathit{sid} \in \mathit{StreamIds}(n,\ m) : \\
&\qquad \forall\, i \in 1\,..\,\mathit{Len}(S(n,\ m,\ \mathit{sid}).\mathit{inq}) : \\
&\qquad\quad \text{LET}\ x \triangleq S(n,\ m,\ \mathit{sid}).\mathit{inq}[i] \\
&\qquad\quad \text{IN}\quad \land\ x.\mathit{kind} = \text{“headers”} \quad \Rightarrow \mathit{Len}(x.\mathit{heights}) \leq \mathit{MaxHeaders} \land \mathit{Contiguous}(x.\mathit{heights}) \\
&\qquad\qquad\quad\ \land\ x.\mathit{kind} = \text{“get-blocks”} \Rightarrow \mathit{Len}(x.\mathit{heights}) \leq \mathit{MaxBlocksPerRequest} \\
&\qquad\qquad\quad\ \land\ x.\mathit{kind} = \text{“blocks”} \quad \Rightarrow \mathit{Len}(x.\mathit{heights}) \leq \mathit{MaxBlocksPerRequest}
\end{aligned}
$$

Honest peers never earn a misbehavior penalty.

$$
\begin{aligned}
&\mathit{NoHonestPenalty} \triangleq \\
&\quad \forall\, n \in \mathit{InitialPeers} : \forall\, m \in \mathit{OtherPeers}[n] : \mathit{nodes}[n].\mathit{score}[m] = 0
\end{aligned}
$$

No interleaving of conformant behaviour ends in a protocol error. Checked in configurations with no $\mathit{Byzantine}$ peers, where any close other than $\mathit{OBSOLETE}$ is a disagreement between honest peers about the draft’s rules.

$$
\begin{aligned}
&\mathit{NoHonestProtocolError} \triangleq \\
&\quad \forall\, n \in \mathit{InitialPeers} : \forall\, m \in \mathit{OtherPeers}[n] : \\
&\qquad \mathit{nodes}[n].\mathit{close}[m] \in \{\mathit{NoCode},\ \text{“OBSOLETE”}\}
\end{aligned}
$$

Accountability: an honest receiver fires $\mathit{PROTOCOL\_ERROR}$ only after a genuine violation by the peer. Tolerated mischief (unknown stream types, unknown handshake records) never sets the ghost flag, so a receiver that punished it would violate this invariant.

$$
\begin{aligned}
&\mathit{CloseAccountable} \triangleq \\
&\quad \forall\, n \in \mathit{InitialPeers} \setminus \mathit{ByzantinePeers} : \forall\, m \in \mathit{OtherPeers}[n] : \\
&\qquad \mathit{nodes}[n].\mathit{close}[m] = \text{“PROTOCOL\_ERROR”} \Rightarrow \mathit{nodes}[n].\mathit{violated}[m]
\end{aligned}
$$

Every genuine violation eventually ends the connection.

$$
\begin{aligned}
&\mathit{EventuallyPunished} \triangleq \\
&\quad \forall\, n \in \mathit{InitialPeers} : \forall\, m \in \mathit{OtherPeers}[n] : \\
&\qquad \Box(\mathit{nodes}[n].\mathit{violated}[m] \Rightarrow \Diamond(\mathit{nodes}[n].\mathit{conn}[m] = \text{“closed”}))
\end{aligned}
$$